

Assignment 4 Report

Software Development for Robotics

Group: PPD2

V. Malaxa s2726254, R. Díaz s3681548

March 11, 2026

4.1 - Package Inheritance Hierarchy

The main goal of this assignment is to learn the principles of inheritance as well as the visualization of it through hierarchy diagrams. For this cause, the first task to be completed was to write an object-oriented program that creates a list of packages with different shipping options. Figure[1] displays the hierarchy diagram created to represent the various types of packages to be implemented. Note how the function `calculate_Cost()` is defined in the parent class and overridden only in the "OvernightPackage" class:

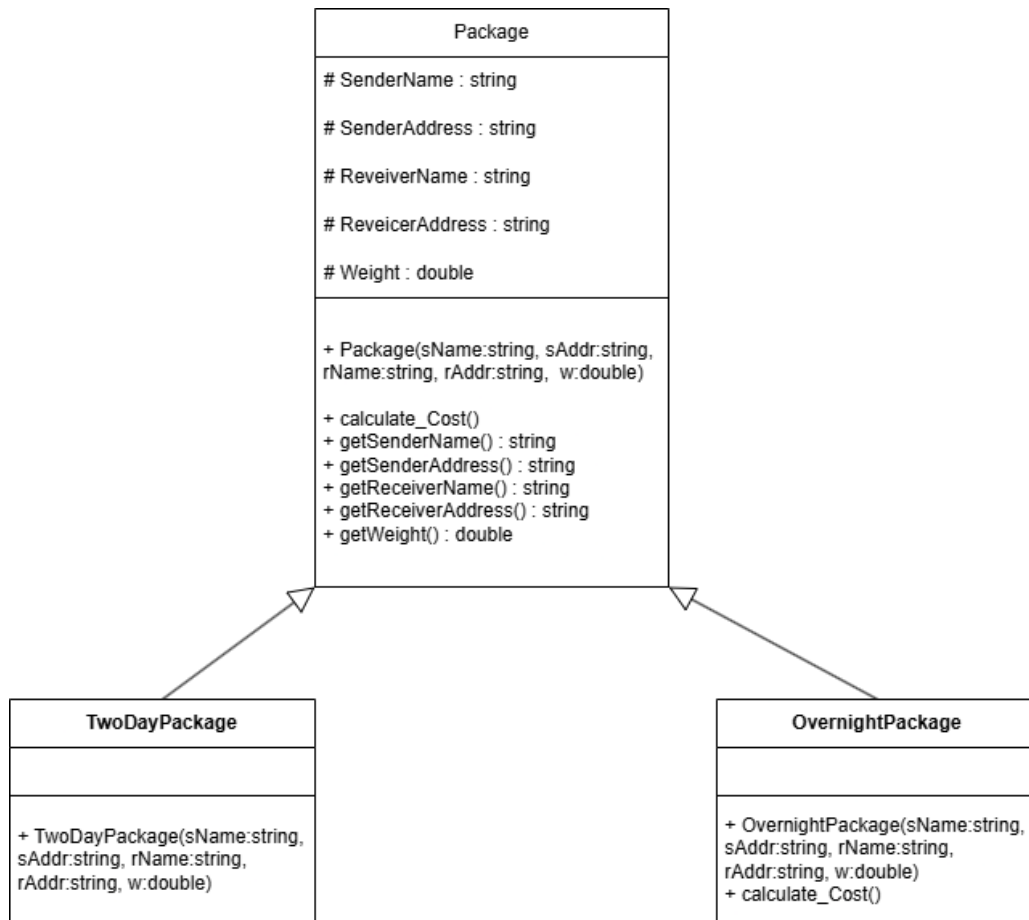


Figure 1: Hierarchy Diagram for various types of packages

The code starts with the declaration of the "**Package**" class. This is the baseline and parent class for both delivery options. The names and addresses of both sender and receiver as well as the weight of the package were declared as protected members of the **Package** class, so that its child classes can access these attributes while preventing direct access from outside the class hierarchy.

In the public section of this parent class, the constructor declaration is included (the string type was used for names and addresses, and the double type was used for storing weight with precision). Inside the public section was also included "`calculate_Cost()`" so that it could be called outside this class, the function used to calculate the cost of the packages. This function was included in the parent "**Package**" class and was defined as a virtual function so that child classes may override it to calculate their own package price. Finally, it is worth noting that inside the public section of the parent "**Package**" class, getter functions were included in the case that protected attributes needed to be accessed from outside the "**Package**" class or its child classes.

The "TwoDayPackage" class inherits all attributes and functions from the "Package" class and uses `calculated_Cost()` exactly as defined in its parent class without modifying it.

The "OvernightPackage" class also inherits all attributes and functions from its parent class "Package", but it overrides the `calculate_Cost()` function in order to include the specified additional cost.

In the `main.cpp` file, the function `print_Costs()` was declared, which makes use of the previously described public getter functions in order to access and then print the sender and receiver addresses, as well as the weight of the package.

In the main loop, a vector of pointers to "Package" objects is created so that different types of packages (TwoDayPackage and OvernightPackage) can be stored. After calling the `print_Costs()` function and displaying the information on the screen, the vector objects are deleted to free allocated memory by calling the virtual destructors for each package. Since the destructor for the "Package" class is declared as virtual, the correct destructor is called for each derived object.

Edge Cases

The edge cases that have been tested out in Figures 2 and 3, with the first two serving as examples of the normal cases. Firstly the weight was tested at the value zero, which resulted in the cost of 5 euro. This is in line with the formula of `calculated_Cost()`, that applies a flat fee regardless of the weight.

Next a very large weight was used, to see if the code faces any numerical issues. There was no issue encountered because of the double type used for the weight, which allows for a wider range of values.

Next a standard test was performed, in which names and addresses were then omitted. This would be useful to see if the code can still return the shipping cost, as well as the "label" only with partial information. It was seen that no errors in the output were encountered with this test. The only variable that needs to have a value is the weight. Without a value, the code will not output any cost or shipping information, as the function `calculate_Cost()` cannot be performed, thus affecting the `print_Cost()` function.

Lastly a negative weight was tested. Logically this would be impossible, but the `calculate_Cost()` function should have no trouble computing the cost. It was found out that the overnight package ended up having a positive value because of the squared weight term inside the calculation.

```

PS C:\Users\vladm\OneDrive\Documents\GitDocs\SDfR\G2Assignment4\G2Assignment4.1_edgecase> .\assignment
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 2 kg
Shipping cost: 10 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 3 kg
Shipping cost: 22.4 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 0 kg
Shipping cost: 5 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 0 kg
Shipping cost: 5 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 10000 kg
Shipping cost: 25005 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 10000 kg
Shipping cost: 1.10025e+08 euro
-----
Sender: | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 3 kg
Shipping cost: 12.5 euro
-----
Sender: | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: 3 kg
Shipping cost: 22.4 euro
-----
Sender: Richie | Address:
Receiver: Vlad | Address: Street 2
Weight: 3 kg
Shipping cost: 12.5 euro
-----
Sender: Richie | Address:
Receiver: Vlad | Address: Street 2
Weight: 3 kg
Shipping cost: 22.4 euro
-----
Sender: Richie | Address: Street 1
Receiver: | Address: Street 2
Weight: 3 kg
Shipping cost: 12.5 euro

```

Figure 2: Enter Caption

```

-----
Sender: Richie | Address: Street 1
Receiver:      | Address: Street 2
Weight: 3 kg
Shipping cost: 12.5 euro
-----
Sender: Richie | Address: Street 1
Receiver:      | Address: Street 2
Weight: 3 kg
Shipping cost: 22.4 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address:
Weight: 3 kg
Shipping cost: 12.5 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address:
Weight: 3 kg
Shipping cost: 22.4 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: -20 kg
Shipping cost: -45 euro
-----
Sender: Richie | Address: Street 1
Receiver: Vlad | Address: Street 2
Weight: -20 kg
Shipping cost: 395 euro
-----

```

Figure 3: Enter Caption

4.3 - Questions

(a)

Inheritance helps model relationships between different classes. Code reuse is also part of inheritance, where a child class can absorb its parent class' data and behaviors and sometimes enhance it, such as the case with one of our child classes "TwoDayPackage" inheriting the function "calculate_Cost()" from its parent class "Package" and expanding upon it to calculate its own shipping cost. The capability of reusing data and functions from the "Package" class to its child classes not only helps us in writing more efficient code by avoiding duplication, we also obtain a better structured cleaner code.

(b)

The characteristics of each access specifier are the following:

- Public members can be accessed from outside the class.
- Private members can only be accessed inside the class where they are declared.

- Protected members cannot normally be accessed from anywhere outside the class, however unlike private members they can be accessed from inherited classes.

(c)

Inheritance creates a new class by using the properties and methods of an existing class, composition works as a special type of aggregation that implies ownership of the added component. For example in the context of a computer:

Inheritance uses an **is-a** relationship. A new class is created based on an existing class and inherits its properties and methods. We would create a general class "computer". This general class would have certain attributes such as "CPU" or "RAM" as well as methods or functions like "boot()" or "shutdown()" that are general to all computer types. Child classes would exist that inherit these general attributes and methods from the general class, and expand upon these. In the case of computers, two possible child classes would be "Laptop" and "Quantum computer". Although these two are both a computer and they both inherit this class' attributes and methods, they would each expand on with their own attributes and methods, for example, a laptop could have an attribute such as "Battery Life" while a quantum computer could have a very different attribute such as "NumberofQubits".

In the case of composition, we are using a **has-a** relationship. In composition, a class is built by using objects of other classes as its components. In the case of computers, this could be explained as the general class being "Computer" and by using composition, we would state that a computer has other objects such as a "Processor", a "Keyboard" and a "Screen". The computer class would contain objects from all of these classes and use them to carry out its functions.

(d)

During program execution, the compiler inserts calls to destructors when objects reach the end of their lifetime. This happens in our program in two situations: When dynamically allocated objects are deleted, a call to a destructor is inserted in the compiler. Figure[1] shows how we create objects using "new" in our main() function, and the compiler inserts destructor calls once "delete" is executed.

Listing 1: snippet of main().cpp where both "new" and "delete" are executed

```

1 packages.push_back(new TwoDayPackage(
2     "Eva", "Street 5",
3     "Frank", "Street 6",
4     5.0));
5
6 print_Costs(packages);
7
8 for (auto p : packages) {
9     delete p;
10 }
```

The other case where destructor calls are inserted in the compiler would be destructors for global and static objects being automatically called when the program finishes execution.

(e)

A virtual destructor is defined with the 'virtual' keyword in a base class. Their main function is ensuring that when using a derived class, destructors are called in the correct order in the case that one of its objects are deleted via a pointer to its base class: **Derived class destructor first, then base class destructor**. Using this correct order is important because it prevents memory leaks as well as undefined behavior from happening.

Without using a virtual destructor, only the base class destructor will be called, skipping the derived class destructor and leading to possible problems.

1 AI statement

During the writing of this assignment AI tools(such as ChatGPT) were only used to identify errors in the code. After which the code was reevaluated and rewritten by the user themselves.